AURA BEAT

Emotion-Aware Music Generator & AI Voice Assistant

Prepared by: Project Development Team & Lead Architect
Date: July 2026

1. Executive Summary & Problem Solved

AuraBeat solves a key limitation of traditional music systems: the requirement of active user interaction and manual catalog filtering which ignores real-time psychological states. AuraBeat automates music selection by introducing an **emotion-sensing layer**. The application parses facial expressions, voice sentiment, or text inputs to tailor the streaming experience to the user's emotional state. By building a unified catalog aggregation engine which merges a local db and online APIs, AuraBeat provides a seamless Web3-ready music player with natural voice feedback.

2. Project Uniqueness Matrix

The following matrix summarizes key features that distinguish AuraBeat from standard streaming apps:

System Dimension	Traditional Music Streaming Apps	AuraBeat Solution (Uniqueness)
Mood Detection	Relies entirely on pre-configured static playlists (chill, workout).	**Dynamic Multi-modal Input**: Runs browser-level computer vision models (TinyFaceDetector) and Web Speech sentiment filters to classify emotions in 1.2s.
CORS Streaming	Fails due to browser origin security limits on external media CDNs.	**Reverse Audio Proxying**: A custom Spring Boot proxy streams raw audio bytes, keeping playback running locally.
Conversation Flow	Static text search fields with zero context retention.	**AI Command Integration**: A custom action-parsing layer translates LLM tokens into executable actions (e.g. navigation, playlist creation).
AI Architecture	Requires heavy server nodes or crashes entirely when cloud keys hit limits.	**Double Fallback Strategy**: Integrates a local python (Hugging Face transformer) service and Java pattern rules if APIs are offline.

3. Core Technical Architecture

AuraBeat follows a decoupled **Full-Stack microservice layout** containing three primary layers:

Component Layer	Technologies Used	Key Responsibilities & Implemented Features
Frontend (Client)	React 19, Vite, Tailwind CSS v4, Lucide Icons, Framer Motion	Dynamic UI state; hooks for voice capture (webkitSpeech); face expression smoothing via a 7-reading majority vote filter.

Backend (Spring)	Java 21, Spring Boot 4, Spring Security, Spring Data JPA, Jackson	JWT stateless sessions; BCrypt verification; Catalog aggregating service; Audio-streaming reverse proxy endpoint to bypass CORS.
Database	MySQL, Hibernate (ORM)	7 relational schema tables: users, songs, playlists, playlist_songs, favorites, emotion_history, and chat_history.
AI Services	Google Gemini API, Hugging Face (distilroberta), Python Flask	Dual-sentiment analysis; action parsing ([ACTION:play_song]); local script fallback parser for offline compatibility.

4. Frontend Deep-Dive (React Implementation)

A. Music State Engine (MusicContext.jsx):
Controls global audio playback. Maps standard HTML5 event hooks into properties like currentSong, shuffleList, volume, and repeat modes. Resolves stream playback in a two-stage process: replaces dummy demo track URLs on the fly from Saavn API mappings, and rewrites external CDN source links through local routes (`/api/audio/stream?url=...`) to avoid browser CORS errors.

B. AI Action Manager (AIContext.jsx):
Integrates native window synthesis for text speech outputs. Uses a regex parser block to decompose text vectors: reads `[ACTION:action_name] [PARAM:value]` and triggers React state actions (e.g. `play_song`, `navigate`, `create_playlist`).

C. Webcam Expression Tracker (Dashboard.jsx):
Dynamically downloads `face-api.js` on mount. Deploys TinyFace models to inspect video frames stream. Combats noise using a **Majority Vote Smoothing Buffer** (runs every 1200ms, stores last 7 readings, only commits the emotion when the consensus rate exceeds 3 readings). Saves the logged emotion history to `/api/emotion`.

5. Backend Deep-Dive (Spring Boot Implementation)

A. Stateless Security Configuration (SecurityConfig.java & Filters):
Spring Security forces stateless sessions. Custom JwtAuthenticationFilter captures request headers (`Bearer `), validates them using JwtTokenProvider (signing HMAC-SHA paths), and creates an authentication instance inside SecurityContextHolder.

B. Catalog Caching & De-duplication (CatalogService.java):
Combines records from Local Database search, JioSaavn Dev API, and Audius APIs. Implements memory caches via ConcurrentHashMaps (5-min search results, 1-hr stream URLs expiry). Deduplicates tracks in O(N) using lowercase text mapping of Title + Artist.

C. CORS Audio Proxy (AudioStreamController.java):
An HTTP reverse proxy. Initiates connections to external CDNs on behalf of the client browser, loads raw audio stream bytes, attaches default `Access-Control-Allow-Origin: *` configurations, and flushes output response buffers.

6. Python Flask SERVICE (ai-service/app.py)

DistilRoBERTa Emotion Classifier:
A local Python microservice exposed on port 5000. Leverages Hugging Face's pipeline model (`j-hartmann/emotion-english-distilroberta-base`) to classify incoming chat strings into 7 classes (Joy, Anger, Fear, Sadness, Surprise, Disgust, Neutral). It uses a local regex keywords engine fallback to predict emotions if the DL model fails due to host memory limits. It responds to POST calls at `/api/analyze`.

7. Database Schema & Query Optimization

The MySQL database model (**music_memory**) is mapped using Hibernate JPA:
• **users**: PK `id`, Unique Index `email`, `password` (hashed).
• **songs**: PK `id`, Fields: `title`, `artist`, `emotion` (Indexed for search speed), `genre`, `song_url`.
• **playlists**: PK `id`, FK `user_id` references `users(id)` (On Delete Cascade).
• **playlist_songs**: Junction table mapping composite key `(playlist_id, song_id)`. Many-to-Many.
• **favorites**: PK `id`, FK `user_id`, FK `song_id`. Multi-column Unique Index prevents duplicate likes.

8. Critical System Challenges & Solutions

1. Frontend WebSpeech Stuttering Bug:
Problem: Chrome webkitSpeechRecognition turns off after short pauses.
Solution: Disabled continuous mode, captured `onend` logs, and forced-restarted calls asynchronously using a React reference hook.

2. Multi-Provider Speed Lag:
Problem: Querying third-party audio wrapper APIs took up to 3 seconds.
Solution: Utilized dual ConcurrentHashMap caching queues inside backend Catalog Services, minimizing API calls for repeated queries.